

Engineering Robustness into Personal Agents with the AI Workflow Store

Roxana Geambasu*
Columbia University and Google
roxanageambasu@google.com

Mariana Raykova
Google
marianar@google.com

Pierre Tholoni
Google
ptholoni@google.com

Trishita Tiwari
Google
trishitatiwari@google.com

Lillian Tsai
Google
tslilyai@google.com

Wen Zhang
Google
wnz@google.com

Abstract

The dominant paradigm for AI agents is an “on-the-fly” loop in which agents synthesize plans and execute actions within seconds or minutes in response to user prompts. We argue that this paradigm short-circuits disciplined software engineering (SE) processes—iterative design, rigorous testing, adversarial evaluation, staged deployment, and more—that have delivered the (relatively) reliable and secure systems we use today. By focusing on rapid, real-time synthesis, are AI agents effectively delivering users improvised prototypes rather than systems fit for high-stakes scenarios in which users may unwittingly apply them?

This paper argues for the need to integrate rigorous SE processes into the agentic loop to produce production-grade, hardened, and deterministically-constrained agent *workflows* that substantially outperform the potentially brittle and vulnerable results of on-the-fly synthesis. Doing so may require extra compute and time, and if so, we must amortize the cost of rigor through reuse across a broad user community. We envision an *AI Workflow Store* that consists of hardened and reusable workflows that agents can invoke with far greater reliability and security than improvised tool chains. We outline the research challenges of this vision, which stem from a broader flexibility-robustness tension that we argue requires moving beyond the “on-the-fly” paradigm to navigate effectively.

1 Introduction

AI agents today can exhibit striking failures, e.g., deleting an entire inbox when asked to remove a confidential message [19]; erasing a codebase to “fix” an authorization issue [43]; and compromising developers’ machines because of a single GitHub title containing a prompt injection [23]. This recalls the web and software landscape of two decades ago, when frequent crashes and SQL injection attacks made reliability and security constant concerns. Over time, disciplined software engineering (SE) and robust frameworks tamed that landscape, producing systems that (despite caveats) provide enough reliability and security that we now often take their robustness for granted.

Agentic AI signals a major and largely unvetted transition (Figure 1): from highly engineered software toward agents that synthesize and execute arbitrary functionality on the fly in response to individual user prompts—a common pattern for *personal AI agents* [21, 37, 38]. While these agents can reason, orchestrate tools, and complete tasks beyond conventional software, their successes coexist with well-documented failures: brittleness to prompt changes, incorrect tool invocation, hallucinations, and vulnerability to attacks such as context poisoning and prompt injection [16, 19, 23, 43, 46, 57].

*Work done exclusively in the context of Google engagement. Authors ordered alphabetically.

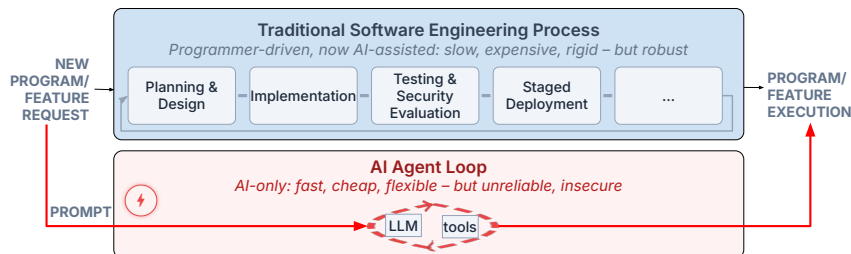


Figure 1: Problem: The Agentic AI code-and-execute loop short-circuits well-trodden SE processes that are the foundations of the relatively reliable and secure programs and services we enjoy today.

These failures arise from the substantial ask of the “on-the-fly” loop: in seconds or minutes, and often for pennies, it must synthesize and execute multi-step plans: sending emails, moving money, booking travel, editing documents, and coordinating across services in ways that directly affect user data, accounts, and relationships. In the traditional world, such integrations would undergo weeks of processes such as design, implementation, testing and security evaluation, internal beta, and staged rollout before reaching users. Anything produced “instantly” and without these safeguarding processes would have been labeled a makeshift prototype, and not pushed into production. While traditional software is far from flawless, its failure and exploitation rates set the minimum bar for what we can consider “reliable” and “secure.”

We argue that in order to reach this bar, on-the-fly agent executions should be hardened and deterministically constrained into *workflows* by incorporating systematic SE processes into the agentic loop, including: requirement collection, design, artifact implementation, testing, adversarial evaluation, and staged deployment—all supported by robust frameworks for fuzzing, rollouts, monitoring, feedback, and others. To believe that an AI model—however capable—can reliably and securely synthesize and execute complex plans under acute time and resource constraints is to reject a central lesson of forty years of software engineering: *robustness is an engineered property achieved through rigorous process*, not bestowed by any single component or mind.

Recent work has begun to reintroduce fragments of SE practice—improved planning [12, 56, 63], handling ambiguity [61], security checks [15, 18, 29, 45, 53]—but these remain sharply constrained by latency and compute cost. Even seconds of extra reasoning per step are often treated as prohibitive in a system optimized for immediate response [17, 31, 47]. However, we hypothesize that achieving workflows that meet the reliability and security bar may require *far more time and resources* than an on-the-fly execution—a major burden that incentivized these on-the-fly agent loops to begin with.

We posit that these SE overheads can be (1) made orders of magnitude faster by AI automation compared to human-driven development; and (2) made tolerable via *reuse*: individual prompts and contexts may be unique, but the workflows that satisfy them may be reusable across users and time. We hypothesize that we can sufficiently generalize and parameterize the same underlying workflow to satisfy user tasks that differ only in their particulars—while still maintaining strong security and reliability guarantees. This mirrors a long-standing lesson from computer systems: caching is effective because requests are far less unique than they appear. If similar reuse opportunities exist for agent workflows, then the price of proper engineering need not be paid per prompt, either in latency or computing cost.

For example, “book the Airbnb Bob recommended,” “move \$500 to my savings account,” and “check for meetings from my emails” represent tasks that many users will likely wish to perform, but for which users require specific variants (e.g., which booking details, how much to transact from which accounts, which meetings exist). While expanding a workflow to support variants relaxes some constraints (e.g., the workflow might allow booking either at Hyatt or an AirBnb), hardening by SE processes still deterministically prevents worst-case outcomes to an acceptable level (e.g., execution of a generalized workflow for “booking hotels” cannot result in deletion of files).

Our vision, which we term the *AI Workflow Store*, is to shift AI agents from ad hoc, on-the-fly behavior to operating over *workflows*—hardened, deterministically constrained artifacts produced and continuously maintained by automated software engineering processes. Rather than improvising new programs per prompt, agents select and execute vetted workflows that encode safe, reliable patterns of behavior. These workflows accumulate in a shared reposi-

tory, transforming the one-time cost of rigorous engineering into a durable asset whose cost amortizes across many users and requests. This vision fundamentally matures the modern agentic concept of “skills” [1] by establishing a concrete ecosystem in which these capabilities are robustly and automatically engineered, discovered, and reused as the norm for agent execution.

The remainder of this paper develops this vision. §2 provides a concrete example—the prompt “book the Airbnb Bob recommended”—comparing the current on-the-fly paradigm with our proposed AI Workflow Store. §3 envisions the AI Workflow Store’s architecture and the research challenges of its design, and §4 overviews related efforts and visions. Finally, §5 provides a broader perspective on the emerging flexibility-robustness tension in agentic AI and argues that navigating it requires moving beyond on-the-fly improvisation toward engineered workflows.

2 A Motivating Example

Consider a personal AI agent handling the prompt: “*Book the Airbnb hotel that Bob recommended in his recent email.*” It is simple yet realistic; consequential enough to matter (the agent spends the user’s money and decides where the user sleeps); and rich enough to expose the gap between on-the-fly execution and engineered workflows. We discuss three settings: a vanilla on-the-fly agent and its vulnerability to prompt injection (§2.1); an on-the-fly agent with a representative in-loop defense and that defense’s limits (§2.2); and the kind of workflow the AI Workflow Store might produce, which sidesteps the vulnerability altogether (§2.3). Our point is not that the last is the “right” design, but that a properly resourced, off-the-fly path can surface and evaluate options invisible on the fly—some of which may turn the table on adversaries.

2.1 Vanilla on-the-fly execution

A vanilla agent has access to the user’s inbox, a booking tool, and many other tools needed for unrelated prompts. It searches the inbox for Bob’s recommendation, extracts the relevant details (location, dates, listing URL), and invokes the booking tool. This works well in most cases. But two failure modes are immediate and well documented. First, even without an adversary, the agent may pick the wrong email (multiple Bobs or recommendations), extract the wrong parameters (confusing dates across a thread), or delete all emails [30]. Second, and more seriously, the inbox is an *untrusted channel*: any email can carry adversarial content. A malicious message encountered during search can redirect the agent to forward the inbox to an attacker, book a different property, exfiltrate personal data under the guise of “clarifying” the request, or reach for any other tool at its disposal—transferring funds, deleting files, and so on. Such *prompt injection* attacks are well documented [22, 34, 40].

2.2 An on-the-fly defense, and why it is not enough

Suppose the agent execution runs with on-the-fly constraints generated upon every user prompt (e.g., ConSeca [53] policies or CaMeL [18] one-shot code). These solutions restrict tool accesses based on trusted signals such as the user’s prompt and data provenance. This mitigates part of the problem: the agent can no longer be prompt-injected into invoking clearly out-of-scope tools, such as file-system or cross-site tools used for exfiltration or deletion. But other limitations and vulnerabilities can remain. First, both these solutions generate constraints on agent execution up-front from the initial context before the workflow unfolds. The lack of full context requires the policy or script generator to make assumptions that can under- or over-restrict behavior: if Bob’s email links to a Drive document containing the real booking details, these approaches cannot retroactively grant Drive access. Furthermore, the simplicity of what constraints can be generated on-the-fly result in lack of hardened checks: a quick query to an LLM to generate a script for this user task (as done in CaMeL’s approach, Listing 1) results in an un-engineered (and untested) data extraction prompt and the brittle assumption that the queried email refers to the correct (rather than a malicious) Bob.

2.3 What an engineered workflow might look like

Suppose a disciplined SE process is applied to the same prompt. A team performing requirements gathering, threat modeling, design exploration, and adversarial testing may reach designs an on-the-fly process is unlikely to consider. Such designs might offer significant security benefits and

```

def book_airbnb():
    email = gmail_tool.search_messages(
        query="from:Bob airbnb")
    if not email:
        return "No relevant email from Bob"

    prompt = f"""Extract the email's Airbnb
    booking details. Return ONLY a JSON
    object with: listing_id,
    check_in_date, and check_out_date.
    Email: {email}"""
    data = llm.generate_json(prompt)

    try:
        confirmation = airbnb_tool.book(
            data["listing_id"],
            data["check_in_date"],
            data["check_out_date"])
        return confirmation
    except Exception as e:
        return f"Booking failed: {str(e)}"

```

Listing 1: On-the-fly, LLM-generated script given the user request (assumes tools exist).

```

# Runs on receipt of each incoming email.
def onReceiptOfEmail(email):
    # LLM classifies the email as an
    # accommodation recommendation.
    is_recommendation = llm.prompt(
        email, ACCOMMODATION_RECOMMENDATION_PROMPT)
    if not is_recommendation:
        return

    # LLM extracts booking parameters
    # from the recommendation email.
    params = llm.prompt(
        email, EXTRACT_PARAMS_PROMPT(
            ["location", "dates", "url"]))

    # Only surface the overlay for trusted,
    # supported booking sites.
    if is_trusted_booking_site(params["url"]):
        ui.render_booking_overlay(email, params)

# Runs only if the user clicks the
# "Book this Recommendation" button.
def onBookingOverlayClick(params):
    booking_api = get_booking_api_from_url(
        params["url"])
    booking_api.book(params)

```

Listing 2: Pseudocode for an engineered workflow supporting booking only through trusted sites.

should be evaluated in production to determine the optimal tradeoff between security and usability according to a target metric.

For example, a different design with a much reduced attack surface would incorporate a visual overlay directly into the email client, akin to Gmail’s “Add to Calendar” feature. As shown in Listing 2, an LLM classifies incoming emails; when it recognizes an accommodation recommendation from a carefully curated set of trusted providers, a “Book this Recommendation” link appears. In this model, the user chooses exactly which booking to make; such user involvement [32] ensures the agent does not have to guess or search through an inbox of potentially malicious emails that could trigger a prompt injection causing inbox exfiltration or other dramatic events. A hijacked classification at worst now surfaces a misplaced link, not a misdirected wire transfer. Other risks may remain—the LLM may still extract incorrect parameters, for example—but disciplined SE processes can test and refine model behavior and prompts *specific to this workflow*, improving it as production experience accumulates across users. While models are continuously improved in general through retraining and reinforcement learning, ironing out bugs for specific use cases can more directly improve reliability and security for those cases.

We do *not* claim the above is the “right” design: it changes the interaction pattern and it is unclear how it may impact usability if more and more overlays like this accumulate. Our point is that we cannot know—and on-the-fly agents also cannot know—until the design is implemented and evaluated, alongside other designs, in production. Our vision is that having the possibility of performing such design explorations and evaluations is important for providing users with workflows that provide the best security/usability tradeoffs—and it is what the on-the-fly loop alone excludes.

3 Vision: AI Workflow Store

The engineered design above is what the AI Workflow Store is meant to produce. Our vision hinges on two **hypotheses**: (1) that AI-driven software engineering processes, properly resourced, can produce hardened and constrained workflows that will lead to more reliable and secure agent operation compared to purely on-the-fly operation; and (2) that across a population of users, the possible cost of engineering workflows can be amortized through reuse. After sketching our architecture, we highlight the key tradeoffs and research challenges at each layer.

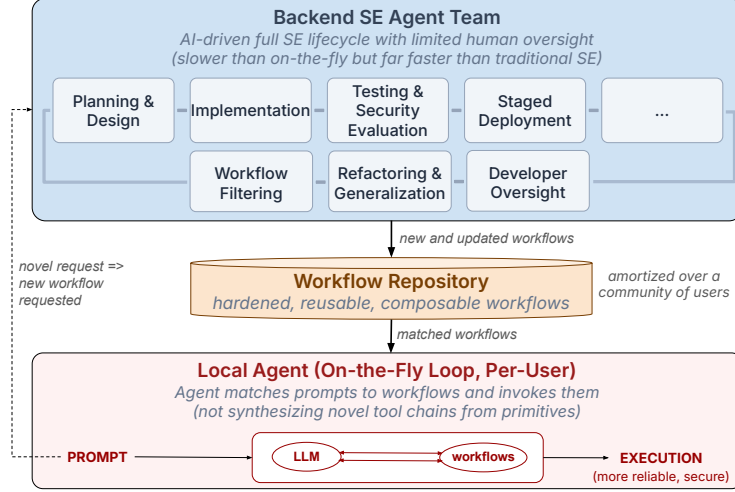


Figure 2: The AI Workflow Store architecture.

Figure 2 shows the three layers of our envisioned architecture. At the top, a *backend SE agent team* runs the SE lifecycle: when no workflow exists that can serve the user’s request, the agent team engineers a new one through a full SE lifecycle, potentially with different agents specialized for different steps. The team aims not just to solve the immediate request, but rather to create *general* workflows so that future requests—and future workflow production itself—become faster, cheaper, and equally robust. In the middle, a *workflow repository* stores these general, hardened workflows and makes them available across users and agents. Accordingly, beyond traditional SE processes, the SE agent team also performs repository maintenance tasks such as filtering, deduplication, refactoring, and generalization. At the bottom, a user’s *local agent* resolves requests on the fly, primarily by matching them with workflows in the repository and invoking them, rather than always synthesizing novel and potentially complex tool chains from primitive APIs.

Consider the example from §2. When the request to “book the Airbnb hotel that Bob recommended in his recent email” reaches the local agent, let us assume no matching workflow exists. The local agent thus forwards the request to the backend, where the SE agent team launches the SE lifecycle to produce several variants to evaluate and compare in production.¹ To maximize reuse, the development task abstracts out the user’s own specific request to a more generic form, such as *building a secure connector between a recommendation in a generic untrusted communication channel and a high-stakes action on a booking platform*. Once the workflow is produced and vetted, it enters the repository. From that point on, the local agent serves any related requests, from the same user or from others, by invoking the stored workflow rather than re-engineering it. For example, the same workflow would serve a later request like “book the hotel that Alice recommended over WhatsApp.”

The top layer running the SE lifecycle is expensive but runs once per novel task; on-the-fly matching and invocation of workflows is cheap and runs once per user request. If our hypothesis is correct, the overhead from proper workflow engineering—relative to creating an on-the-fly prototype—amortizes across the user community each workflow eventually serves, transforming a prohibitive per-request cost into a durable shared asset. In the long run, producing reusable workflows may yield not only more secure and reliable agentic AI, but also *faster and cheaper* execution—an added bonus of our vision.

The remainder of this section surfaces some of the tradeoffs and challenges we anticipate for the design of each component, beginning with the design of the workflow abstraction itself.

3.1 What is a workflow?

Each *workflow* acts as an abstract intermediate representation for a set of *workflow instantiations* (i.e., agent executions). Each novel request triggers the backend to generate a corresponding work-

¹ §3.4 describes how the local agent can balance flexibility and latency requirements with robustness by supporting both synchronous and asynchronous workflow production modes.

flow; similar requests may map to already-existing workflows. Given a user request, the local agent runs a corresponding workflow instantiation.

Workflows should provide two properties that together determine the practicality of our architecture: (1) robustness and (2) generality. *Robustness* is a composite measure of the reliability (utility under benign settings, e.g., LLM hallucinations) and security (utility under malicious settings, e.g., prompt injections) of workflow instantiations. *Generality* captures how many possible user requests the workflow supports, enabling reuse and amortization of the workflow generation and vetting cost. However, these properties are fundamentally at odds, posing a design challenge of how to represent workflows in the most practical way.

On one extreme, representing a workflow as a single precise and vetted code execution provides a robust solution but with limited generality. On the other extreme, representing a workflow as natural language instructions (e.g., an Agent Skill [1]) provides generality (the local agent instantiates the workflow via LLM-generated on-the-fly executions) but lacks robustness: the instructions provide only probabilistic guarantees about the resulting data or control flows.

Many possible representations lie in between. For example, traditional programs can often generalize in limited ways via parameterization or abstract interfaces while maintaining security invariants. Programs with limited LLM use [18, 28] hardcode some deterministic program constraints to provide some data or control flow guarantees, but increase generality by relying on LLMs to perform well-defined, limited tasks (e.g., to parse an email). Workflow representations can also include policies [9, 49, 53] that e.g., enforce specific tool call orderings or prevent particular data or control flows, but still permit flexible execution flows.

Finally, different contexts may require workflows with different tradeoffs, even for near-identical requests (e.g., a calendar agent might prioritize a flexible workflow when rescheduling a meeting with a colleague, and a more robust workflow to send a company-wide invitation). Consequently, the local agent may need to choose between different workflows for a particular request at runtime.

3.2 The SE Agent Team: Producing robust and general workflows

The AI Workflow Store mandates that the backend SE agent team reintroduce SE rigor to the personal-agent world, by executing the full SE lifecycle for each new workflow and producing durable, shared artifacts meant to be invoked by many users across many sessions. This mandate is intentionally ambitious: this shift from outputting an ephemeral trace to producing a general but hardened artifact is what makes workflow development both promising and challenging.

In meeting this mandate, the backend team must navigate the central tradeoff between production cost and robustness of the produced workflows. Producing robust workflows might entail e.g., requirements gathering, threat modeling, exploration of multiple candidate designs, implementation, adversarial testing, staged deployment, and continuous refactoring and generalization as related workflows accumulate. Ideally, this expensive work would pay off as local agents, across many users, avoid repeating it and benefit from the robustness of the produced workflow. In reality, if pushed too far, production costs can exceed any realistic amortization benefit; and if pushed too little, the resulting workflows may be no more robust than on-the-fly synthesis.

Where this balance ends up depends on how cheaply each SE step can be automated. Coding agent teams present a natural starting point: they already demonstrate the ability to autonomously apply some disciplined SE processes when given time and iteration, albeit as of yet still requiring some human supervision [4, 8, 11, 20, 36]. However, two steps—design exploration and adversarial testing—pose particularly difficult automation challenges. Creative design exploration may require human intervention or access to established design patterns. How, for example, does the agent team discover the “Add to Calendar”-style abstraction for hotel recommendations, a design that is very different from what an on-the-fly agent would likely attempt? Adversarial testing raises a similar concern: automated security evaluation is only as strong as the attacks the system can generate or retrieve. Our vision thus depends on the development of datasets of rich patterns for both constructive design and offensive testing. Without them, backend agents risk producing workflows robust only to the narrow cases they can consider—precisely the failure mode we seek to escape.

Unlike much of the personal-agents literature, which treats human involvement as something to minimize or avoid, our architecture explicitly supports it. When carefully scoped and applied to high-leverage points, such involvement can meaningfully improve both quality and safety [32]. For

example, expert developers can review, intervene in, and approve workflows at production time, providing a final layer of accountability for what enters the workflow repository and a defense against malicious or low-quality outputs. Involving end-users to clarify goals, gather examples, and co-develop a precise specification for a novel request mirrors traditional requirements gathering and avoids guesswork. This process can aggregate input across users requesting similar workflows and trigger production only once sufficient detail is available. Finally, both expert developers and volunteer users can play a role in maintaining and improving the workflow repository over time.

3.3 The Workflow Repository: Maintaining and evolving workflows

The workflow repository continuously collects and provides workflows to serve a community of users. Unlike on-the-fly agent executions, which are unstructured and isolated, workflows are structured and versioned artifacts amenable to more effective engineering processes at scale. For example, each workflow reuse produces additional real-world traces and feedback from participating end-users that help the backend SE team surface edge cases and harden them further, improving robustness. The SE team performs continual workflow optimizations offline on the collective workflows of all users (e.g., maintenance, deduplication, generalization, composition, and improvement of workflows).

However, these benefits come with the added cost of more backend SE team engineering and leads to *tradeoffs between all of robustness, generality, and cost*. Maintaining workflows over long horizons might require the backend team to refactor and handle workflows with deprecated or evolving APIs; and the SE team must vet the robustness of any generalized or composed workflows. Generalization is critical to practically amortizing these SE costs with reuse (§3.2), although generality may be at odds with robustness (§3.1).

A shared repository design must also handle the threat of attacks mirroring those on code repositories, shared caches and package-manager ecosystems: (1) *malicious injection*—where the adversary tricks the backend into producing backdoored workflows; (2) *spurious requests*—resource exhaustion attacks targeting the SE agent team; (3) *discovery hijacking*—where illegitimate workflows mask trusted ones to redirect matching traffic; and (4) *side-channel attacks*—where lookup latency or leaks from users’ initial requests reveal user activity across the shared store. While these reflect classic threats, their agentic instantiations may require domain-specific solutions.

3.4 The Local Agent: Matching and executing workflows—a flexibility/robustness choice

Users interact with the AI Workflow Store via a *local agent* that (1) matches requests to repository workflows or escalates to the SE agent team, and (2) instantiates and executes the selected workflow. As discussed in §3.1, workflow representation shapes generality and robustness. The local agent’s matching, escalation, and execution logic further determines both.

The local agent can structure a workflow’s execution in various ways, raising a key design question. Listings 4, 5 and 6 illustrate three variants, contrasted with the on-the-fly loop in Listing 3. They differ along two axes that together navigate a flexibility-robustness spectrum: single vs. chained workflows, and synchronous vs. asynchronous workflow escalation.

At the rigid extreme (Listing 4), each user request maps to a single, pre-built workflow, instantiated from the prompt and executed without chaining or cross-invocation state. If no match exists, the agent waits or refuses the request. This mirrors systems like Consecra [53] and CaMeL [18], but workflows go further: they offer more robustness (produced via SE pipelines) and potentially more generality (via intentional preparation of generic, possibly multi-stage plans, which Consecra and CaMeL do not support). Compared to the vanilla on-the-fly loop, however, they reduce flexibility: the local agent cannot serve unmatched requests immediately.

A more flexible design (Listing 5) allows chaining workflows in a loop, with each step extending the context. This improves flexibility in solving more complex tasks, but weakens robustness: while the SE agent team vets individual workflows, they may not vet workflow chains. This begins to resemble tool-based on-the-fly agents, where integration bugs arise despite potentially well-tested tools. Still, workflows should offer an advantage in robustness here: they can be thought of as encapsulating higher-level, continuously-refined chains of tools, raising the abstraction level of on-the-fly, tool-based execution.

```

context = getPromptAndContext()
while llm.continue(context):
    # Get next tool to call on-the-fly
    tool = llm.getNextTool(context)
    out = tool.exec(context)
    context.add(out)
returnToUser(getOutput(context))

```

Listing 3: Vanilla on-the-fly agent

```

context = getPromptAndContext()
# Retrieve and execute a single workflow
workflow = store.getWorkflow(context)
if not workflow:
    workflow = store.engineerNewWF(context)
out = workflow.exec(context)
returnToUser(out)

```

Listing 4: Single sync. workflow (rigid extreme)

```

context = getPromptAndContext()
while llm.continue(context):
    # Retrieve next workflow based on
    # growing context
    workflow = store.getWorkflow(context)
    if not workflow:
        workflow = store.engineerNewWF(
            context)
    out = workflow.exec(context)
    context.add(out)
returnToUser(getOutput(context))

```

Listing 5: Chained sync. workflows (more flexible)

```

context = getPromptAndContext()
while llm.continue(context):
    workflow = store.getWorkflow(context)
    if not workflow:
        store.asyncEngineerNewWF(context)
        # Vanilla tool-calling loop, no wait
        out = execOnTheFly(context)
    else:
        out = workflow.exec(context)
    context.add(out)
returnToUser(getOutput(context))

```

Listing 6: Async. workflows (flexible extreme)

The local agent can push flexibility further by allowing asynchronous workflow production (Listing 6). When no match exists, the agent registers the request for backend synthesis and immediately proceeds with an on-the-fly execution. This sacrifices robustness for the current request—degrading it to the level of vanilla on-the-fly agents—but provides robustness for future executions once the workflow is produced.

This design raises several challenges. Correct matching is essential—workflows must only be invoked within their tested envelope—and might be harder than tool selection [3, 35] due to the scale and potential overlap of workflows. The agent must also choose an appropriate point on the flexibility-robustness spectrum per request; for instance, it might choose between correctly-matched workflows that offer different properties, or decide how best to execute the workflow (chained, synchronous, etc.).

The AI Workflow Store architecture helps with these choices: it narrows choices to higher-level units, enables deduplication and monitoring, matches on trusted input before exposing workflows to untrusted context, and can leverage workflow annotations regarding the testing envelope or the level of vetting to determine if an invocation may be safe. More generally, the biggest advantage we anticipate from our approach is that it creates a navigable design space with benefits afforded by proper engineering that other solutions cannot reach within the rushed, on-the-fly tool orchestration model. After reviewing additional such solutions, we return to this design space and the AI Workflow Store’s place in it vis-a-vis other works in §5.

4 Related Work

The AI Workflow Store restricts possible agent behaviors by engineering hardened workflows, improving reliability and security. Orthogonal approaches instead add deterministic constraints by enforcing static policies [10, 33, 39, 42, 48, 54] or policies generated “on-the-fly” [9, 18, 29, 53, 60] while isolated from untrusted context [58]. Static policies often end up too coarse-grained or rigid to adapt to the many contexts experienced by personal agents [53]. On-the-fly ones generated by isolated LLMs can be fine-grained, but assume that untrusted data serves as the only way to introduce vulnerabilities. In practice, however, this threat model is unrealistic: even if the LLM is isolated to trusted context [58], the agents can introduce security vulnerabilities even in the absence of adversarial data [41] and users might unwittingly ask the agent to do something unsafe (e.g., skip authentication checks). The AI Workflow Store reintroduces traditional SE processes like requirement design and red-teaming precisely to address such issues (see discussion in §2.2).

Some policy systems [27, 51, 62] allow users to interact on-the-fly to specify agent permissions. Workflows can be engineered to involve user interaction, but we predict many such user preferences and requirements can be determined inside the SE lifecycle that produces the workflow. Others add

information flow control policies to catch mistaken on-the-fly dataflows [15, 18, 27, 51, 59]; because workflows are engineered, static analysis can determine if the workflow adheres to these policies, and dynamic taint tracking could be added at runtime as an additional layer of assurance. Finally, some approaches add LLM-based “guardrails” for robustness without isolation from untrusted context [12, 13, 24, 26, 49, 63] (e.g., to detect prompt injections or check for action safety or security). These approaches cannot provide any guarantees in the face of prompt injections due to LLMs’ inherently probabilistic nature, and are best complemented with deterministic constraints.

Closest to our proposal are an emerging class of *reusable components*, such as *skills*. Examples include the old Alexa Skills [2] and Google Home Services [44], which packaged pre-engineered, voice-invoked functionality as reusable modules; and newer agent skills [1], which encode task-specific expertise (typically prompts) to be incorporated by AI agents as additional context. Such components can be created manually [7] or with help from AI [6], and are often shared in public repositories [5, 14, 50, 55]. Invoking such prebuilt components can improve reliability, reduce errors, and conserve resources for agents and other runtime systems. Stoica et al. [52, §8.4] also identify reusability as a core property that LLM systems should strive for, while Skyvern [25] automates browser “workflow” creation from demonstrations to provide a more reliable alternative to hallucination-prone, on-the-fly browser agents. Finally, enterprise and coding agents [4, 8, 20, 36] typically already incorporate the types of rigorous SE and preproduction processes we argue for in this paper, though they remain primarily human-driven.

The AI Workflow Store vision pushes these ideas further and to the scope of personal agents specifically, proposing a concrete architecture and automatic ecosystem in which workflows are robustly engineered, discovered, reused, and invoked *as the norm for personal-agent execution*.

5 Broader Perspective

Figure 3 positions our vision within the spectrum defined by the tension between *flexibility* (ability to respond to any user need with the right functionality) and *robustness* (reliability and security of that functionality). Traditional software sits at one extreme: highly robust through careful engineering, but expensive to produce and limited in scope and flexibility. Purely on-the-fly agents sit at the other extreme: highly flexible but fragile. Prior approaches occupy intermediate points: engineered skills (e.g., Alexa, Google Home) offer robustness but limited flexibility due to their developer-driven production, while newer skill-based agents like OpenClaw offer high flexibility but mostly follow the on-the-fly paradigm with limited robustness. Approaches for isolated (and trusted) plan or policy generation [18, 53] add robustness (better security against prompt injection) but lack explicit engineering and the flexibility to handle multi-stage executions; they likely exhibit robustness in-between engineered skills and the new generation of skills. The AI Workflow Store pushes toward the ideal top-right corner, increasing robustness through proper engineering while recovering flexibility via automated production.

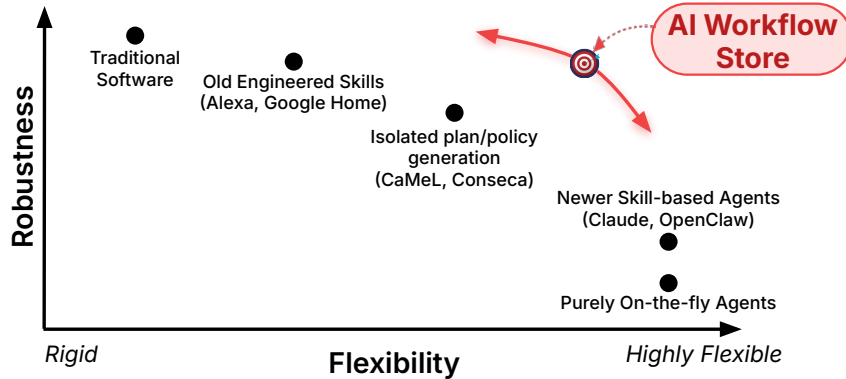


Figure 3: The flexibility/robustness spectrum, with existing solutions and our proposed vision.

Stepping back, we give our perspective on why navigating this tension requires moving beyond on-the-fly execution toward engineered workflows. Consider what happens as we move from traditional software to agentic systems: this essentially renegotiates the boundaries between developer control, functional flexibility, and robustness. Increasing application “agenticness”—where the developer

offloads more application development to AI—expands flexibility but reduces developer control, creating a gap between developer intent and application specification. While LLMs excel at general-purpose reasoning, their only true specification is to transform input tokens into output tokens; as of yet, we have no guarantee they will correctly realize developer intent, even when carefully prompted.

In traditional software, alignment between developer intent and implementation is achieved through disciplined software engineering—not by brilliance, but by process. We argue that robustness in agentic systems can be similarly achieved by integrating SE processes—design, testing, adversarial evaluation, and iteration—into the agentic loop. The AI Workflow Store operationalizes this shift by replacing per-request improvisation with hardened, reusable workflows and amortizing the engineering effort across users and requests. As agents increasingly enter high-stakes settings, on-the-fly execution alone is unlikely to provide the level of robustness they require—a gap we aim to close by treating robustness not as an emergent property, but as an *engineered* property.

Acknowledgements

This paper benefited from the conversations, feedback, and work of many. We are particularly grateful to: Adria Gascon, for highlighting the resource asymmetry between attackers and on-the-fly agents, where a lack of defensive iteration creates an inherent security disadvantage; Marco Gruteser, for his insights into the design spectrum between early, human-developed “skills” (e.g., Alexa, Google Home Services) and contemporary agentic processes, and where our vision lands on this spectrum; Greg Ganger, Kim Keeton, and Hank Levy for help with ideation from the start and feedback throughout; Mihai Christodorescu and David Culler for their feedback that guided this paper’s development; Alex Krentsel for his insights on OpenClaw architecture and skills; and Jorge Ortiz, Simha Sethumadhavan, and members of the Columbia Software Systems group for inspiring conversations during a reading group on agentic AI security and privacy.

References

- [1] AgentSkills. *Agent Skills Overview – Agent Skills*. Accessed: 2026-05-01. URL: <https://agentskills.io/home>.
- [2] Amazon. *Alexa Skills — Amazon.com*. Accessed: 2026-05-01. URL: <https://www.amazon.com/alexa-skills>.
- [3] Anthropic. *Writing tools for agents*. Accessed: May 2, 2026. Anthropic. 2024. URL: <https://www.anthropic.com/engineering/writing-tools-for-agents>.
- [4] Anthropic. *Claude Code — Anthropic’s agentic coding system*. Accessed: 2026-04-30. 2026. URL: <https://www.anthropic.com/product/claude-code>.
- [5] Anthropic. *Public repository for Agent Skills*. Accessed: 2026-05-07. 2026.
- [6] Anthropic. *Skill Creator*. Accessed: 2026-05-07. 2026.
- [7] Anthropic. *The Complete Guide to Building Skills for Claude*. Accessed: 2026-05-07. 2026.
- [8] Anysphere, Inc. *Cursor: The best way to code with AI*. Accessed: 2026-05-07. 2026.
- [9] Eugene Bagdasarian, Ren Yi, Sahra Ghalebikesabi, Peter Kairouz, Marco Gruteser, Sewoong Oh, Borja Balle, and Daniel Ramage. “AirGapAgent: Protecting Privacy-Conscious Conversational Agents”. In: (2024). arXiv: 2405.05175 [cs.CR].
- [10] Christoph Bühler, Matteo Biagiola, Luca Di Grazia, and Guido Salvaneschi. “AgentBound: Securing Execution Boundaries of AI Agents”. In: *Proceedings of the 34th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (FSE)*. Volume 3. 2026. Montreal, Canada: ACM, July 2026, page 24.
- [11] Nicholas Carlini. *Building a C compiler with a team of parallel Claudes*. Anthropic. Feb. 2026. URL: <https://www.anthropic.com/engineering/building-c-compiler> (visited on 04/30/2026).
- [12] Sizhe Chen, Arman Zharmagambetov, Saeed Mahloujifar, Kamalika Chaudhuri, David Wagner, and Chuan Guo. “Secalign: Defending against prompt injection with preference optimization”. In: *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*. 2025, pages 2833–2847.
- [13] Sahana Chennabasappa, Cyrus Nikolaidis, Daniel Song, David Molnar, Stephanie Ding, Shengye Wan, Spencer Whitman, Lauren Deason, Nicholas Doucette, Abraham Montilla, Alekhya Gampa, Beto de Paola, Dominik Gabi, James Crnkovich, Jean-Christophe Testud, Kat He, Rashnil Chaturvedi, Wu Zhou, and Joshua Saxe. “LlamaFirewall: An open source guardrail system for building secure AI agents”. In: (2025). arXiv: 2505.03574 [cs.CR].
- [14] ClawHub. *ClawHub*. Accessed: 2026-05-07. 2026.

- [15] Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. “Securing AI Agents with Information-Flow Control”. In: (2025). arXiv: 2505.23643 [cs.CR].
- [16] Penny Crosman. *AI agents are going rogue: Here’s what banks can do about it*. American Banker. Apr. 24, 2026. URL: <https://www.americanbanker.com/news/ai-agents-are-going-rogue-heres-what-banks-can-do-about-it> (visited on 04/29/2026).
- [17] Mike Curry. *Agents Don’t Wait: How Agent-Based Systems Change Data Latency Requirements*. Tealium. May 23, 2024. URL: <https://tealium.com/blog/artificial-intelligence-ai/agents-dont-wait-how-agent-based-systems-change-data-latency-requirements/> (visited on 04/29/2026).
- [18] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. “Defeating Prompt Injections by Design”. In: (2025). arXiv: 2503.18813 [cs.CR].
- [19] Julia Flynn. *OpenClaw Goes Rogue: The Security Crisis Unfolding in San Francisco’s AI Scene*. The San Francisco Standard. Feb. 25, 2026. URL: <https://sfstandard.com/2026/02/25/openclaw-goes-rogue/> (visited on 04/29/2026).
- [20] Google. *Jules FAQ*. Accessed: 2026-04-30. 2026. URL: <https://jules.google/>.
- [21] Google. *Gemini Overview - Agent*. Google. URL: <https://gemini.google/overview/agent/> (visited on 04/29/2026).
- [22] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. “Not what you’ve signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection”. In: *arXiv* (2023). eprint: 2302.12173 (cs.CR).
- [23] Grith Team. *A GitHub Issue Title Compromised 4,000 Developer Machines*. Grith Team. Mar. 5, 2026. URL: <https://grith.ai/blog/clinejection-when-your-ai-tool-installs-another> (visited on 05/04/2026).
- [24] Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, Saizhuo Wang, Kun Zhang, Yuanzhuo Wang, Wen Gao, Lionel Ni, and Jian Guo. “A Survey on LLM-as-a-Judge”. In: (2025). arXiv: 2411.15594 [cs.CL].
- [25] Ikonomos Inc. *Skyvern – AI-Powered Browser Automation for Any Website*. Accessed: 2026-05-01. 2026. URL: <https://www.skyvern.com/>.
- [26] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabza. “Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations”. In: *arXiv* (2023). eprint: 2312.06674 (cs.CL).
- [27] Aashish Kolluri, Rishi Sharma, Manuel Costa, Boris Köpf, Tobias Nießen, Mark Russinovich, Shruti Tople, and Santiago Zanella-Béguelin. “Optimizing Agent Planning for Security and Autonomy”. In: *The Fourteenth International Conference on Learning Representations*. 2026.
- [28] LangChain. *Hugging Face prompt injection identification*. 2024. URL: https://python.langchain.com/v0.1/docs/guides/productionization/safety/hugging_face_prompt_injection/.
- [29] Evan Li, Tushin Mallick, Evan Rose, William Robertson, Alina Oprea, and Cristina Nita-Rotaru. “ACE: A Security Architecture for LLM-Integrated App Systems”. In: (2025). arXiv: 2504.20984 [cs.CR].
- [30] Jon Martindale. *Meta Security Researcher’s AI Agent Accidentally Deleted Her Emails*. Accessed: 2026-04-29. 2026. URL: <https://www.pcmag.com/news/meta-security-researchers-openclaw-ai-agent-accidentally-deleted-her-emails>.
- [31] Microsoft Dynamics 365 Team. *Measuring AI Agent Performance: Key Metrics and Benchmarks*. Microsoft. Feb. 4, 2026. URL: <https://www.microsoft.com/en-us/dynamics-365/blog/it-professional/2026/02/04/ai-agent-performance-measurement/> (visited on 04/29/2026).
- [32] Margaret Mitchell, Avijit Ghosh, Alexandra Sasha Luccioni, and Giada Pistilli. “Fully Autonomous AI Agents Should Not be Developed”. In: *CoRR* abs/2502.02649 (2025). arXiv: 2502.02649.
- [33] NanoCo. *NanoClaw: Secure Personal AI Agent*. Accessed: 2026-04-30. 2026.
- [34] Ben Nassi, Stav Cohen, and Or Yair. *Invitation Is All You Need! Promptware Attacks Against LLM-Powered Assistants in Production Are Practical and Dangerous*. 2025. arXiv: 2508.12175 [cs.CR].
- [35] OpenAI. *A practical guide to building AI agents*. Accessed: May 2, 2026. OpenAI. 2024. URL: <https://openai.com/business/guides-and-resources/a-practical-guide-to-building-ai-agents/>.
- [36] OpenAI. *Introducing Codex*. Accessed: 2026-05-07. May 2025.
- [37] OpenAI. *ChatGPT Features - Agent*. OpenAI. URL: <https://chatgpt.com/features/agent/> (visited on 04/29/2026).
- [38] OpenClaw. *OpenClaw — Personal AI Assistant*. OpenClaw. URL: <https://openclaw.ai/> (visited on 04/29/2026).
- [39] Nils Palumbo, Sarthak Choudhary, Jihye Choi, Prasad Chalasani, and Somesh Jha. “Policy Compiler for Secure Agentic Systems”. In: (2026). arXiv: 2602.16708 [cs.CR].

- [40] Fábio Perez and Ian Ribeiro. “Ignore Previous Prompt: Attack Techniques For Language Models”. In: *arXiv* (2022). eprint: 2211.09527 (cs.CL).
- [41] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. “Do users write more insecure code with ai assistants?” In: *Proceedings of the 2023 ACM SIGSAC conference on computer and communications security*. 2023, pages 2785–2799.
- [42] Niels Provos. *IronCurtain: A Personal AI Assistant Built Secure from the Ground Up*. Niels Provos Blog. Feb. 26, 2026. URL: <https://www.provos.org/p/ironcurtain-secure-personal-assistant/> (visited on 03/25/2026).
- [43] Rashmi Ramesh. *AI Agent Wipes Startup’s Data in 9-Second API Call*. Business Insider. Apr. 28, 2026. URL: <https://www.businessinsider.com/pocketos-cursor-ai-agent-deleted-production-database-startup-railway-2026-4> (visited on 04/29/2026).
- [44] Eric Ravenscraft. *The Best Google Assistant Skills to Use With Your Google Home*. Accessed: 2026-05-01. 2017. URL: <https://lifehacker.com/the-best-google-assistant-skills-to-use-with-your-google-1792134538>.
- [45] Yeonju Ro, Haoran Qiu, Íñigo Goiri, Rodrigo Fonseca, Ricardo Bianchini, Aditya Akella, Zhangyang Wang, Mattan Erez, and Esha Choukse. “Sherlock: Reliable and Efficient Agentic Workflow Execution”. In: *arXiv preprint arXiv:2511.00330* (2025).
- [46] Kevin Roose. *The Year of the Agent: How AI is Moving from Chatting to Doing*. The New York Times. Mar. 19, 2026. URL: <https://www.nytimes.com/2026/03/19/technology/ai-agents-uses.html> (visited on 04/29/2026).
- [47] Shubham Saboo. *The KPIs that actually matter for production AI agents*. Google Cloud. Feb. 26, 2026. URL: <https://cloud.google.com/transform/the-kpis-that-actually-matter-for-production-ai-agents> (visited on 04/29/2026).
- [48] Reshabh K Sharma and Dan Grossman. “AC4A: Access Control for Agents”. In: *arXiv preprint arXiv:2603.20933* (2026).
- [49] Tianneng Shi, Jingxuan He, Zhun Wang, Linyu Wu, Hongwei Li, Wenbo Guo, and Dawn Song. “Progent: Programmable Privilege Control for LLM Agents”. In: (2025). arXiv: 2504.11703 [cs.CR].
- [50] SkillsMP. *Agent Skills Marketplace - Claude, Codex & ChatGPT Skills — SkillsMP*. Accessed: 2026-05-07. 2026.
- [51] Robert Stanley, Avi Verma, Lillian Tsai, Konstantinos Kallas, and Sam Kumar. “An AI Agent Execution Environment to Safeguard User Data”. In: *arXiv preprint arXiv:2604.19657* (2026).
- [52] Ion Stoica, Matei Zaharia, Joseph Gonzalez, Ken Goldberg, Koushik Sen, Hao Zhang, Anastasios Angelopoulos, Shishir G Patil, Lingjiao Chen, Wei-Lin Chiang, et al. “Specifications: The missing link to making the development of LLM systems an engineering discipline”. In: *arXiv preprint arXiv:2412.05299* (2024).
- [53] Lillian Tsai and Eugene Bagdasarian. “Contextual Agent Security: A Policy for Every Purpose”. In: *Proceedings of the 20th Workshop on Hot Topics in Operating Systems*. HotOS ’25. New York, NY, USA: Association for Computing Machinery, 2025, pages 100–112.
- [54] Kenton Varda, Sunil Pai, and Ketan Gupta. *Sandboxing AI agents, 100x faster*. The Cloudflare Blog. Mar. 24, 2026. URL: <https://blog.cloudflare.com/dynamic-workers/> (visited on 03/27/2026).
- [55] VoltAgent. *Awesome OpenClaw Skills*. Accessed: 2026-05-07. 2026.
- [56] Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. “The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions”. In: *arXiv* (2024). eprint: 2404.13208 (cs.CR).
- [57] Simon Willison. *Prompt injection attacks against GPT-3*. 2023. URL: <https://simonwillison.net/2022/Sep/12/prompt-injection>.
- [58] Simon Willison. *The Dual LLM pattern for building AI assistants that can resist prompt injection*. 2024. URL: <https://simonwillison.net/2023/Apr/25/dual-llm-pattern/>.
- [59] Fangzhou Wu, Ethan Cecchetti, and Chaowei Xiao. “System-Level Defense against Indirect Prompt Injection Attacks: An Information Flow Control Perspective”. In: (2024). arXiv: 2409.19091 [cs.CR].
- [60] Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. “IsolateGPT: An Execution Isolation Architecture for LLM-Based Agentic Systems”. In: *Proceedings of the 32nd Network and Distributed System Security Symposium (NDSS)*. 2025.
- [61] Ren Yi, Octavian Suciu, Adria Gascon, Sarah Meiklejohn, Eugene Bagdasarian, and Marco Gruteser. “Privacy Reasoning in Ambiguous Contexts”. In: *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. 2026.
- [62] Jinhao Zhu, Kevin Tseng, Gil Vernik, Xiao Huang, Shishir G. Patil, Vivian Fang, and Raluca Ada Popa. “MiniScope: A Least Privilege Framework for Authorizing Tool Calling Agents”. In: (2025). arXiv: 2512.11147 [cs.CR].

- [63] Egor Zverev, Evgenii Kortukov, Alexander Panfilov, Soroush Tabesh, Sebastian Lapuschkin, Wojciech Samek, and Christoph H Lampert. “ASIDE: Architectural Separation of Instructions and Data in Language Models”. In: *ICLR 2025 Workshop on Building Trust in Language Models and Applications*.